AI

ALA-3

AIM: Basic Chatbot Creation

Students must perform this in group of 2 by building a rule-based chatbot using if-else conditions on a selected topic. The chatbot should respond based on user input. Submit a Python (.py) file with chatbot code and a Word/PDF document with example interactions. Upload both files on the GMIU Portal.

Solution:

Project Report: Rule-Based Library Chatbot

Student 1 Name: Aayushi Sanghrajka

Student 2 Name: Sarvaiya Vaidehiba

Subject: AI

Topic Selected: Library

1. Project Overview

This project involves the design, development, and deployment of a web-based rule-based chatbot built for a college or public library system. The objective is to automate routine library enquiries regarding operating hours, book issuance, membership rules, fines, and campus locations.

2. Sample Chatbot Conversations

The table below documents sample user test inputs and the corresponding rule-based outputs generated by the codebase:

Test Case	User Input	Chatbot Response Output
1	"Hello"	"Hello! Welcome to the Library Enquiry Chatbot. How can I help you?"
2	"What are your opening hours?"	"The library is open from 9:00 AM to 5:00 PM on working days."
3	"Is a book available?"	"Please provide the book title or ask the librarian to check availability."
4	"How can I borrow a book?"	"You can borrow books using your valid library membership card."
5	"What is the fine for late return?"	"A late return may incur a fine. Please check the library's current fine rules."
6	"Where is the library located?"	"The library is located on the college campus. Please check with the office for directions."
7	"Can I use a computer?"	"Please ask the librarian about computer and internet facilities."
8	"What is your name?"	"Sorry, I didn't understand. You can ask about timings, books, issuing, returns, fines, or membership."

3. System Architecture & Python Code Implementation (app.py)

The application logic uses Python string-matching methods (.lower().strip() and substring evaluations) inside a modular function library_chatbot(message) to parse JSON payload requests from the frontend and deliver precise, rule-based responses.

4. Deployment Architecture & Summary

- **Repository Structure:** Organized with root-level app.py, requirements.txt (containing flask and production WSGI server dependencies), a templates / directory containing index.html, and a static/ directory containing style.css.
- **Hosting Configuration:** Successfully deployed as a live cloud application on Render utilizing gunicorn for process management, ensuring global accessibility via the submitted live link.